

System execution and silicon

Slide 1 · 0-2 min

System execution and silicon

Every technique must discharge a named verification claim.

Choose where to run → Interpret software evidence → Use implementation-level simulation

Teaching script

Open with a decision the audience must make in this area. Ask which result they currently treat as sufficient evidence and keep that answer for the closing discussion.

The lesson selects a practical argument from the book. The study handout preserves the detailed examples, citations and worked artifacts.

Source: ch17:13-43, ch18:13-62, ch19:15-42, ch20:18-51

Learning outcomes

Slide 2 · 2–5 min

Learning outcomes

Demonstrate these decisions on a real plan row.

- Choose an execution platform with a usable oracle.
- Resolve model disagreements without default authority.
- Interpret GLS and power-aware results within their scope.
- Turn a silicon escape into upstream evidence.

Teaching script

Read the outcomes as observable tasks. Ask learners to identify which one they can already demonstrate and which one currently depends on undocumented judgment.

At closure, require a concrete artifact or explanation for an outcome rather than a general expression of confidence.

Source: ch17:13–43, ch18:13–62, ch19:15–42, ch20:18–51

Route through the lesson

Slide 3 · 5–7 min

Route through the lesson

Four blocks, each ending with a decision.

- Choose where to run
- Interpret software evidence
- Use implementation-level simulation
- Bring silicon evidence upstream

Teaching script

Allow eighteen minutes for each block: a concept, its engineering consequence, a worked interpretation and a short discussion. Keep the discussion tied to the evidence available in the example.

Reserve the final eleven minutes for the assigned exercise and application to the learners' work. The timing is a course design allocation, not an engineering estimate.

Source: ch17:13–43, ch18:13–62, ch19:15–42, ch20:18–51

More cycles answer only some questions

Slide 4 · 7–11 min

More cycles answer only some questions

Select the platform from the scenario and its oracle.

- Distinguish many short runs from one long execution.
- Budget observability as well as speed.

Teaching script

Contrast broad seed exploration with firmware that must execute deeply before reaching its interesting state. Faster cycles do not replace scenario breadth or an effective checker.

Ask what failure evidence the chosen platform can retain. Platform selection made without a debug plan can move the bottleneck from execution to diagnosis.

Source: ch17:13–43, ch17:44–72

The host boundary can dominate

Slide 5 · 11–16 min

The host boundary can dominate

Transaction batching matters when work crosses platforms.

Host test → Transaction boundary → Hardware execution

Teaching script

Walk one transaction from host-side software through a transactor to the hardware execution platform. Frequent fine-grained interaction can consume the benefit of fast hardware cycles.

Choose a boundary that preserves the verification intent and amortizes communication. Record any behavior moved into the transactor that now needs its own validation.

Source: ch17:73–92, ch17:93–220

Capacity is a shared project resource

Slide 6 · 16–21 min

Capacity is a shared project resource

Image preparation and debug access belong in the schedule.

- Include compile, queue and turnaround time.
- Allocate platform capacity by verification value.

Teaching script

Have learners distinguish raw cycle rate from usable throughput for a team. Build time, image availability and access contention can dominate the time to a useful answer.

A platform resource policy should identify the runs that need it and the evidence they return. A simulator workload is not automatically a good emulation workload.

Source: ch17:221–327

Discussion: accelerate the wrong workload

Slide 7 · 21–25 min

Discussion: accelerate the wrong workload

Name the evidence that would settle the decision.

- A fast platform spends most time waiting for host calls.
- The scenario itself needs few cycles.
- What boundary or platform decision should be revisited?

Teaching script

Expected answer: inspect the transaction partition and communication granularity, and reconsider whether this scenario benefits from that platform at all.

Do not prescribe batching blindly: verify that a changed boundary preserves the observations and timing behavior the test is meant to check.

Source: ch17:13–43, ch17:93–220

Prototypes trade visibility for execution

Slide 8 · 25–29 min

Prototypes trade visibility for execution

A long run needs a reproducible failure record.

- Plan observability before the failure arrives.
- Keep software, image and configuration together.

Teaching script

Ask which internal observation would distinguish the likely hypotheses after a software-visible failure. On a prototype, that observation may need to be planned before the run.

Preserve the complete context of the experiment. A software log without the corresponding hardware image and setup can leave a long failure impossible to reproduce.

Source: ch18:13–62

A virtual model has an abstraction boundary

Slide 9 · 29–34 min

A virtual model has an abstraction boundary

Fast software progress does not validate omitted timing.

- Use the model for the behavior it represents.
- Resolve disagreements against agreed intent.

Teaching script

A virtual model can be useful for software bring-up while omitting timing and implementation details relevant to hardware verification. State that scope before interpreting a passing software run.

When model and RTL disagree, neither wins by being called the reference. Return to the specification and identify whether hardware, software expectations or the specification need correction.

Source: ch18:63–93

A hybrid boundary is part of the model

Slide 10 · 34–39 min

A hybrid boundary is part of the model

Timing-dependent behavior can invalidate an attractive partition.

- Review synchronization and interaction frequency.
- Retain system checking beyond one driver trajectory.

Teaching script

Consider a host virtual platform coupled to RTL. A partition that crosses a tight timing dependency may be expensive or semantically inappropriate even when both halves work independently.

A successful firmware driver follows one programmed trajectory. It does not replace verification of the full legal peripheral behavior or the interactions that trajectory never exercises.

Source: ch18:94–146, ch18:147–249

Discussion: who owns the mismatch?

Slide 11 · 39–43 min

Discussion: who owns the mismatch?

Name the evidence that would settle the decision.

- A virtual peripheral and RTL disagree.
- Each team declares its artifact authoritative.
- What decision should triage produce?

Teaching script

Expected answer: resolve the intended behavior and classify a hardware defect, software/model-side defect or specification gap, rather than choosing an artifact by organizational ownership.

For a specification gap, identify the authority that must settle intent. Neither implementation team can close that gap by silently selecting its own interpretation.

Source: ch18:63–93, ch18:250–351

GLS samples implementation behavior

Slide 12 · 43–47 min

GLS samples implementation behavior

Static timing analysis remains the timing authority.

- Select runs for implementation-specific risks.
- Keep timing annotation and configuration reproducible.

Teaching script

Explain the complementary questions: static timing evaluates timing paths under its constraints; gate-level simulation exercises selected functional scenarios with implementation semantics.

A passing sample does not cover every constrained path. Use risk to select GLS runs instead of treating the RTL regression list as an automatic implementation-level campaign.

Source: ch19:15–42, ch19:43–90, ch19:91–160

Unknowns can make a checker silent

Slide 13 · 47–52 min

Unknowns can make a checker silent

Inspect the comparison semantics before accepting a clean run.

- An unknown comparison can bypass a failure branch.
- Check initialization, models and the observer.

Teaching script

Use the chapter's diagnostic distinction between an actual design problem, a modeling issue and an observer that does not reject unknown data. A result can appear clean because the checker evaluated an unknown condition.

Do not repair the waveform first and assume the verification is sound. Confirm that the checking policy would expose the unacceptable unknown in the relevant observation window.

Source: ch19:43–90

Power intent adds temporal obligations

Slide 14 · 52-57 min

Power intent adds temporal obligations

Isolation and restoration must protect the observation boundary.

- Check corruption, isolation and state restoration.
- Exercise the selected power transitions.

Teaching script

Walk shutdown and wake-up in event order. Isolation must protect receivers before corruption reaches them and remain effective until restored behavior is safe to observe.

The simulator's treatment of power states and corruption is part of the model. An optimistic state mapping can make the trace look valid while omitting the failure mechanism.

Source: ch19:161-320, ch19:321-434

Discussion: clean output during power-off

Slide 15 · 57-61 min

Discussion: clean output during power-off

Name the evidence that would settle the decision.

- A powered-down block remains readable in the model.
- Its outputs stay legal throughout shutdown.
- What modeling premise must be challenged?

Teaching script

Expected answer: whether the selected power-aware setup actually models corruption, isolation and restoration for the relevant states.

A clean trace is meaningful only if the model admits the failure being checked. Ask for an observation showing that the intended corruption and protection mechanisms were active.

Source: ch19:161-320

Trace storage fixes a tradeoff

Slide 16 · 61–65 min

Trace storage fixes a tradeoff

More observed signals leave fewer cycles in a fixed buffer.

Signal selection → Fixed trace storage → Retained history

Teaching script

Keep the argument qualitative before opening the chapter's trace-depth calculation. The finite buffer is a physical resource chosen before the failure is known.

Select signals that separate likely hypotheses, and plan triggers that place the interesting interval inside the retained window. Wider tracing without that plan can shorten useful history.

Source: ch20:18–51, ch20:52–97

Bring-up is an evidence ladder

Slide 17 · 65–70 min

Bring-up is an evidence ladder

A failed step is interpretable only after its prerequisites hold.

- Establish clocks, reset and access first.
- Advance through checks with explicit dependencies.

Teaching script

Ask why a failed memory test cannot immediately be assigned to the memory subsystem if basic access or initialization is still unverified.

Each bring-up stage should establish premises for the next. Preserve the first failing observation and the configuration rather than proceeding until some later application happens to run.

Source: ch20:98–134

An escape owes an upstream change

Slide 18 · 70–75 min

An escape owes an upstream change

A workaround alone leaves the verification gap open.

- Localize by configuration and time.
- Create a pre-silicon reproducer where feasible.
- Update the plan, model or checker that missed it.

Teaching script

Separate immediate containment from explaining the escape. Configuration bisection can reduce the hypothesis space before expensive temporal localization.

A hardware or firmware workaround needs its own verification. Then identify which earlier observation would have found the problem and make that lesson durable in the verification flow.

Source: ch20:135–201, ch20:202–239, ch20:240–374

Discussion: a successful workaround

Slide 19 · 75–79 min

Discussion: a successful workaround

Name the evidence that would settle the decision.

- Firmware avoids the failing operating mode.
- The product now survives the customer scenario.
- Which verification work remains?

Teaching script

Expected answer: validate the workaround's scope and side effects, retain the original failure, and repair the upstream verification gap.

Containment can be necessary without demonstrating root cause or preventing recurrence. Ask for the concrete plan row, test or checker change that closes the learning loop.

Source: ch20:202–239, ch20:240–374

Exercise: resolve a model disagreement

Slide 20 · 79–85 min

Exercise: resolve a model disagreement

Choose evidence before choosing an owner.

- A virtual peripheral and RTL disagree on a register side effect.
- State the possible triage outcomes.
- Explain what a shared description fixes and what it cannot fix.

Teaching script

Exercise 18.2 (judgement). A peripheral's virtual model and its RTL disagree on a register side effect, and each team offers its own artifact as the reference. Decide what the disagreement establishes about which artifact is wrong, using the argument §18.2 gives against the model that becomes the specification by default. State the three outcomes §18.5 requires triage to reach instead, and which of them leaves the defect owned by neither team. Name the structural fix §18.2 takes from Chapter 15, and the one class of error that neither the fix nor the comparison can surface.

Assign book Exercise 18.2 in full. The answer must include the specification-gap outcome and the limitation of common-source generation.

Expected answer: disagreement alone does not identify the wrong artifact. Resolve hardware, software/model expectations or a specification gap. Generating consistent artifacts from one description prevents drift, but cannot expose a shared error in that description.

Source: ch18:63–93, ch18:250–351

What to change on Monday

Slide 21 · 85–87 min

What to change on Monday

Make one existing verification decision reviewable.

- Record the boundary of one virtual model.
- Audit an unknown-sensitive checker.
- Trace a workaround to its upstream verification change.

Teaching script

Ask each learner to choose one of these actions in an existing project. Require a small, inspectable deliverable and identify the person who will review it.

Keep the change connected to the lesson's evidence limits. A new dashboard, script or model is useful only if it strengthens the claim the plan row is meant to establish.

Source: ch17:13–43, ch18:13–62, ch19:15–42, ch20:18–51

Scope and next reading

Slide 22 · 87–90 min

Scope and next reading

Use the handout for the detailed artifacts and sources.

- Read the System execution and silicon handout and book Chapters 17–20.
- Platform setup recipes and full power-intent syntax remain in the handout.

Teaching script

Revisit the outcomes and ask the room to state a limitation of the strongest result discussed today. A complete answer identifies a model boundary, an assumption, a missing oracle or the scope of the underlying evidence.

Assign the detailed reading to the relevant chapters. The omitted topics are explicit scope boundaries, not claims that those topics are unimportant.

Source: ch17:13–43, ch18:13–62, ch19:15–42, ch20:18–51